Alexandria University, Faculty of Engineering
Specialized Scientific Programs
Natural Language Processing
Instructor: Prof. Dr. Ayman Khalafallah
Assigned: 23/02/2025
Due: 08/03/2025



Assignment Overview

In this assignment, you will work on two primary tasks:

1. Building your first language model based on N-grams, using an unsupervised subset of the IMDB reviews dataset. You will evaluate the model using the perplexity metric and analyze the results.

2. Performing sentiment classification into five distinct classes using the Stanford Sentiment Treebank (SST) dataset. You will employ two different classification approaches: Naive Bayes and Logistic Regression. Additionally, you will generate a confusion matrix and derive performance metrics from it.

Part 1: N-gram Language Model

Objective

An N-gram language model is one of the earliest and simplest approaches to language modeling. In this section, you will construct an N-gram model step-by-step.

Dataset: IMDB Reviews

We will use the unsupervised portion of the IMDB reviews dataset. This dataset consists of a large number of movie reviews collected from the Internet Movie Database (IMDB). For this task, you only need the text and not any sentiment labels.

Steps and Requirements

1. **Data Preprocessing:**
   - Tokenize the text into words.
   - Apply text normalization steps such as lowercasing, removing punctuation, etc.
2. **Constructing the N-gram Model:**
   - Build a unigram model as a starting point (optional but helpful for understanding).
   - Extend the model to bigrams, trigrams, or a chosen N-gram size.
   - Implement Laplace smoothing to handle unseen N-grams.
3. **Evaluation: Perplexity:**
   - Split the dataset into training and testing sets.

Eng. Mariem Saeed **Eng. Mohamed Salama**
Eng. Mazen Yaser **Eng. Youssef sameh**

- Train your model on the training set, then compute perplexity on the testing set.
- Compare different N-gram settings.

You should follow the provided starter code in this notebook for an outline of the basic steps in reading and preprocessing the IMDB data.

Part 2: Text Classification

Dataset: Stanford Sentiment Treebank (SST)

The dataset is sourced from the Stanford Sentiment Treebank (SST), a collection of movie reviews labeled with sentiment scores (a real value) and some other annotations that are irrelevant to our use case here.

Preprocessing for Classification

Each text sample has a sentiment score in the range [0, 1], which you will convert into one of five discrete classes:

- From 0 to 0.2 (0.2 included) → class 0: "*very negative*"
- From 0.2 to 0.4 (0.4 included) → class 1: "*negative*"
- From 0.4 to 0.6 (0.6 included) → class 2: "*neutral*"
- From 0.6 to 0.8 (0.8 included) → class 3: "*positive*"
- From 0.8 to 1.0 (1.0 included) → class 4: "*very positive*"

Part 2.1: Naïve Bayes Classification

Algorithm Implementation (From Scratch)

Implement a Naive Bayes classifier using only NumPy (no extra libraries). Follow the theoretical steps carefully, including:

- Calculating priors for each class.
- Calculating the likelihood of each token (word) given a class.
- Inferring a label for unseen text by maximizing the posterior probability.

Comparison with scikit-learn

After finishing your own implementation, reproduce the same classification results using `scikit-learn`. A straightforward approach:

- Use `CountVectorizer` to transform text data into a bag-of-words representation.
- Use `MultinomialNB` for the classification step.

You may need to tune some parameters to match your from-scratch results exactly. Use your functions from Part 3 (evaluation metrics) to compare the two approaches.

Eng. Mariem Saeed **Eng. Mohamed Salama**
Eng. Mazen Yaser **Eng. Youssef sameh**

Part 2.2: Logistic Regression

Feature Representation

In this section, generate features based on word bi-grams. For example, consider the sentence:

"I love this movie very much"

The word bi-grams are:

$$(\text{I}, \text{love}),\ (\text{love}, \text{this}),\ (\text{this}, \text{movie}),\ (\text{movie}, \text{very}),\ (\text{very}, \text{much}).$$

Construct a vocabulary of all unique bi-grams in your dataset. Each sentence is then represented by a vector of length equal to the number of unique bi-grams, with a 1 if the bi-gram occurs in the sentence and a 0 otherwise (this is a sparse representation).

Algorithm Implementation (From Scratch)

Implement the logistic regression algorithm from scratch using only NumPy, following the standard formulation:

- Initialize weights and bias.
- Use a suitable learning rate and (mini-)batch stochastic gradient descent.
- Update weights iteratively by minimizing the cross-entropy loss.

Remember to shuffle your data each epoch to maintain the stochastic nature of SGD, and be mindful of memory constraints (you may need to delete unnecessary variables).

Comparison with scikit-learn

After implementing your version, compare its performance with:

- `sklearn.linear_model.LogisticRegression`
- `sklearn.linear_model.SGDClassifier` (with settings that make it act like logistic regression).

Discuss which approach performs better and why, and which one is more comparable to your from-scratch implementation.

Part 2.3: Confusion Matrix & Evaluation Metrics

Implementation (From Scratch)

1. Implement a function to generate a confusion matrix, given predictions and ground-truth labels.
2. From this confusion matrix, compute:
   - Precision (per class and macro-averaged)
   - Recall (per class and macro-averaged)
   - F1 score (per class and macro-averaged)
3. Do **not** use any libraries except NumPy.

Eng. Mariem Saeed **Eng. Mohamed Salama**
Eng. Mazen Yaser **Eng. Youssef sameh**

Comparison with scikit-learn

Compare your confusion matrix and derived metrics with the results of:

- `sklearn.metrics.confusion_matrix`
- `sklearn.metrics.precision_score`, `sklearn.metrics.recall_score`, `sklearn.metrics.f1_score`, etc.

Extra Notes

- You have limited memory in Colab ($\sim$ 12 GB). Be mindful of data types to avoid unnecessary precision (e.g., do not use `float64` if `float32` is sufficient).
- You may need to manually free memory in Python using the `del` command in your training loop.
- Shuffling data each epoch is essential to maintain randomness in SGD.
- Remember to vectorize operations for efficiency rather than relying on Python `for` loops.

Grading Policy

This project should be completed in teams of three. Although the assignment may be challenging, it is designed to provide hands-on experience.

Deadline

All submissions are due by **08/03/2025**.

Submission

You must submit zip file named id1_id2_id3_asg1.zip containing the two Jupyter notebooks (`.ipynb`) containing:

- Code for all parts.
- Documentation and explanations of your approach.
- Final results and any comparison plots or tables.

Eng. Mariem Saeed **Eng. Mohamed Salama**
Eng. Mazen Yaser **Eng. Youssef sameh**